You can use the `Run` extension method to build a simple middleware component that always generates a response. This extension takes a single lambda function that runs whenever a request reaches the component. The `Run` extension always generates a response, so no middleware placed after it ever executes. For that reason, you should always place the `Run` middleware last in a middleware pipeline.

TIP Remember that middleware components run in the order in which you add them to the pipeline. If a middleware component handles a request and generates a response, later middleware never sees the request.

The `Run` extension method provides access to the request in the form of the `HttpContext` object you saw in chapter 4. This contains all the details of the request in the `Request` property, such as the URL path, the headers, and the body of the request. It also contains a `Response` property you can use to return a response.

The following listing shows how you could build a simple middleware component that returns the current time. It uses the provided `HttpContext` context object and the `Response` property to set the `Content-Type` header of the response (not strictly necessary in this case, as `text/plain` is used if an alternative content type is not set) and writes the body of the response using `WriteAsync(text)`.

Listing 31.1 Creating simple middleware using the `Run` extension

```
app.Run(async (HttpContext context) =>                ❶
{
    context.Response.ContentType = "text/plain";      ❷
    await context.Response.WriteAsync(                ❸
        DateTimeOffset.UtcNow.ToString());            ❸
});
```